

Scriptony — Architektur-Analyse

Plan vs. Ist: Trennung von narrative Struktur und temporaler Realisierung

Datum: 2026-05-13 | Repo: scriptony-appwrite

1. Zusammenfassung der aktuellen Architektur

Scriptony verfügt über drei getrennte View-Layer, die aktuell über den `TimelineStateContext` als Single Source of Truth verbunden sind:

Ansicht	Komponenten	Zweck
Dropdown	<code>FilmDropdown</code> , <code>AudioDropdown</code> , <code>BookDropdown</code>	Hierarchische Planung: Akt → Sequenz → Szene → Shot / Track
Native	<code>NativeScreenplayView</code> , <code>NativeAudiobookView</code> , <code>NativeBookView</code>	Format-Ansicht wie echtes Drehbuch / Hörspiel-Script
Timeline	<code>VideoEditorTimeline</code> , <code>AudioTimeline</code>	Visuelle Zeitleiste mit Zeitachse, Längen & Lane-Positionierung

Der `TimelineStateContext` hält aktuell alle Daten (Acts, Sequences, Scenes, Shots, Clips, Characters) und bietet Undo/Redo über einen History-Stack.

2. Bewertung der Synchronisationsfragen

2.1 Dropdown ↔ Native synchron KORREKT

Beide Ansichten rendern **denseelben Strukturbaum** — nur in unterschiedlichem Layout. Der Native-View ist ein *Read-Mostly-Renderer* für denselben State, auf den das Dropdown schreibt. Das passt vollständig.

2.2 Dropdown ↔ Timeline synchron KONZEPTIONELL PROBLEMATISCH

Die beiden Views bedienen **unterschiedliche Domänen**:

- **Dropdown / Native** = *Narrative Struktur* (logisch, hierarchisch)
„Szene 3 kommt nach Szene 2 und enthält 4 Dialoge.“

- **Timeline** = *Temporale Realisierung* (zeitlich, linear)
 „Szene 3 startet bei 02:34, dauert 1:45, Dialog 1 beginnt bei 02:40.“

Das Problem: Hard-Synchronisation zwingt die Struktur-Darstellung, Zeit-Informationen zu führen, die sie nicht braucht. Umgekehrt zwingt sie die Timeline, nur Struktur-Operationen zuzulassen (z. B. kann ein Shot in der Timeline nicht frei vor eine Szene geschoben werden, weil der Dropdown das nicht abbilden kann).

Empfehlung: Die Timeline sollte die Struktur **referenzieren**, aber nicht identisch spiegeln. Sie braucht eine eigene Zeit-Layer.

2.3 Der „Plan vs. Ist“-Gedanke BESTE INTUITION

Die richtige architektonische Trennung:

DROPDOWN / NATIVE = „PLAN“

Narrative Struktur

- Was existiert? (Akte, Szenen, Dialoge, SFX-Vorhaben)
- In welcher Reihenfolge?
- **Keine Zeiten, keine Pixel-Längen**

TIMELINE = „IST“

Temporale Realisierung

- Wann startet was? (`startSec`)
- Wie lang dauert was? (`durationSec`)
- Frei verschiebbar auf Zeitachse
- **Kann von der Plan-Reihenfolge abweichen!**

Die Timeline sollte **nicht** den Dropdown-State direkt mutieren, wenn eine Szene zeitlich verschoben wird. Stattdessen:

- **Plan** bleibt unverändert in Dropdown/Native
- **Ist** wird in der Timeline verschoben (oder geschnitten, gestreckt)

Ein „**Compare-Modus**“ würde dann die Plan-Reihenfolge neben der Ist-Reihenfolge anzeigen und Abweichungen markieren.

2.4 Soundeffekte dialogübergreifend LÖSBAR

Im Dropdown sind Tracks unter einer Szene gruppiert. Ein Regen-Sound oder Musik läuft aber oft über mehrere Szenen.

Lösung: Zeitbasierte Referenz statt hierarchischer Einbettung

Ein Audio-Track hat zwei Darstellungsebenen:

1. **Im Dropdown:** Er wird unter der Szene angezeigt, in der er *erstmalig vorkommt* (sein „Ursprung“). Er hat ein Feld `plannedStartSceneId` und `plannedEndSceneId` (oder `plannedDuration`).
2. **In der Timeline:** Er wird an seiner tatsächlichen `startTime` auf der SFX-/Music-Lane platziert. Ob er über Szene-Grenzen hinausläuft, ist dort irrelevant.

Das ist ein **Cross-Cutting-Element** — die hierarchische Ansicht kann es nur approximativ darstellen, die Timeline exakt.

2.5 Dialogboxen innerhalb von Szenen verschieben

KORREKT — ABER NUR IN DER TIMELINE

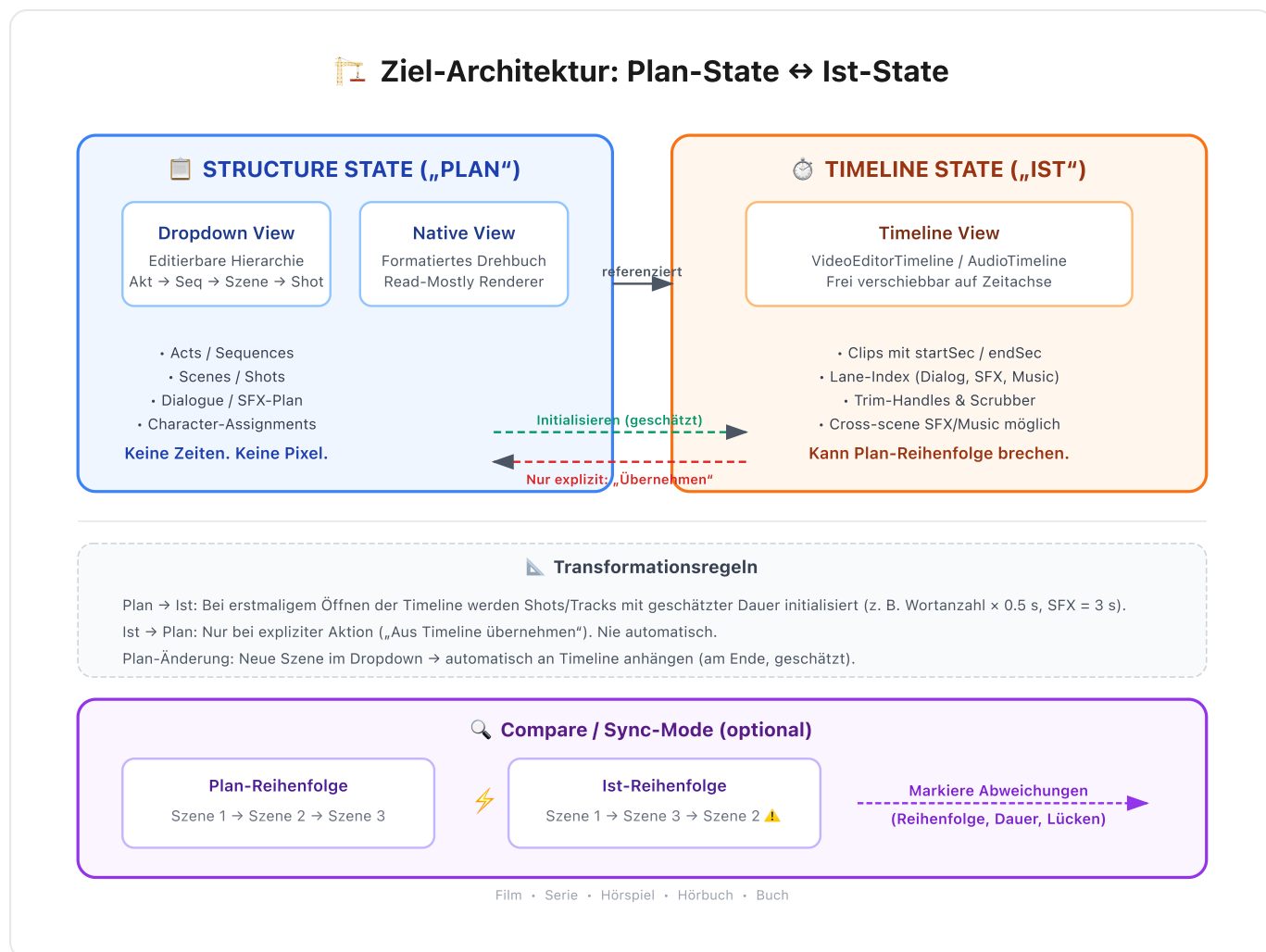
Ja, Dialogboxen sollen innerhalb einer Szene verschiebbar sein — aber **nur in der Timeline**, über **relative Positionierung**:

- Szene hat in der Timeline: `startSec`, `durationSec`
- Ein Shot/Dialog darin hat: `relativeStartSec` (von Szene-Anfang), `durationSec`
- Wenn Szene verschoben wird:
$$\text{shot.startSec} = \text{scene.startSec} + \text{shot.relativeStartSec}$$

In der **Dropdown-Ansicht** gibt es nur eine **Listen-Reihenfolge** (Shot 1, Shot 2, Shot 3), die die narrative Reihenfolge definiert. „Verschieben innerhalb einer Szene“ gibt es dort nicht.

Operation	Dropdown (Plan)	Timeline (Ist)
Shot-Reihenfolge ändern	Ja (narrative Reihenfolge)	Nein — beeinflusst Zeit
Shot zeitlich verschieben	Nicht möglich (keine Zeit)	Ja (<code>startSec</code>)
Szene verschieben	Nicht möglich (keine Zeit)	Ja (<code>startSec</code> + Inhalt verschiebt sich relativ)

3. Visuelle Architektur-Skizze



4. Konkrete Architektur-Empfehlungen

A. Trenne Plan-State von Ist-State

Aktuell existiert ein monolithischer `TimelineStateContext`. Langfristig empfiehlt sich ein **zweispuriges Modell**:



Die vorhandenen `Clip`-Typen mit `startSec/endSec` sind bereits Schritt 1 in diese Richtung. Sie sollten jedoch **entkoppelt** von der harten Shot-Reihenfolge werden.

B. Definiere klare Transformationsregeln

Richtung	Regel
Plan → Ist (Initialisierung)	Shots/Tracks werden mit geschätzter Dauer belegt (z. B. Wortanzahl × 0.5 s, SFX-Vorschlag = 3 s).
Ist → Plan (Rückfluss)	Nur bei expliziter Aktion (z. B. „Aus Timeline übernehmen“). Nie automatisch.
Plan-Änderung (neue Szene im Dropdown)	Automatisch in der Timeline anhängen (am Ende, mit geschätzter Dauer).
Ist-Änderung (Clip in Timeline ziehen)	Nie den Plan verändern.

C. Der „Compare-Modus“ als vierte Ansicht

Statt drei synchronisierten Views:

1. **Dropdown** = Editierbarer Plan
2. **Native** = Lesbarer Plan
3. **Timeline** = Editierbare Ist-Zeit
4. **Compare/Sync-View** = Zeigt Plan neben Ist, markiert Abweichungen

Beispiel: Plan sagt Szene 3 ist die 5. Szene im Akt. Ist zeigt Szene 3 wurde in der Timeline vor Szene 2 geschoben → Markierung „**Reihenfolgeabweichung**“.

D. Audio / Hörspiel konkret

Der `AudioDropdown` zeigt Tracks bereits unter Szenen — gut für die Planung.

Für **dialogübergreifende SFX**:

- **Im Dropdown:** Zeige SFX unter der Szene, wo er *startet*. Füge `crossesIntoNextScenes: boolean` oder `plannedDuration: number` hinzu. Über-Szenen-Grenzen: Hinweis-Indikator (z. B. Pfeil nach rechts).

- **In der Timeline:** Der Track liegt auf der SFX-/Music-Lane an `startTime` mit echter `duration` . Keine Begrenzung auf Szene-Grenzen.

5. Riskante Annahmen in der aktuellen Implementierung

Was ich sehe	Risiko
<code>VideoEditorTimeline</code> berechnet Blöcke aus der Struktur-Hierarchie (<code>calculateActBlocks</code> , <code>calculateSequenceBlocks</code>)	Zwingt die Timeline, exakt die Plan-Struktur abzubilden. Eine Szene kann nicht zeitlich vor eine andere geschoben werden, wenn sie im Plan später kommt.
<code>Clip</code> ist an <code>shotId</code> gekoppelt	Gute Trennung beginnt, aber wenn man einen Clip frei verschiebt (über Shot-Grenzen hinweg), bricht die Konsistenz zur Shot-Reihenfolge.
<code>TimelineStateContext</code> hat Undo/Redo über alle Daten	Ein Undo in der Timeline (Zeitverschiebung) wirkt auch auf den Dropdown zurück. Das ist für den User verwirrend, da Zeit und Struktur semantisch unterschiedliche Aktionen sind.

6. Fazit

Deine Idee	Bewertung
Dropdown = Planungsinstanz	KORREKT
Native = visuelles, greifbares Drehbuch	KORREKT
Timeline = Produktionsinstanz (Zeit, Längen)	KORREKT
Dropdown ↔ Native synchron	JA, BEIDE SIND PLAN-RENDERER
Dropdown ↔ Timeline nicht direkt synchron	DEIN BESTER INSTINKT
„Plan vs. Ist“-Vergleichsmodus	SEHR MÄCHTIGE NÄCHSTE STUFE
SFX dialogübergreifend	LÖSBAR MIT ZEIT-ATTRIBUTEN + DROPDOWN-ANZEIGE UNTER START-SZENE
Dialogboxen in Szene verschieben (Timeline)	JA, MIT RELATIVER ZEITPOSITIONIERUNG ZUR SZENE

💡 **Kern-Erkenntnis:** Deine Intuition ist richtig. Die drei Ansichten sollten **nicht** alle identisch synchron sein. Plan (Dropdown/Native) und Ist (Timeline) sollten getrennt werden, mit einer klaren „Plan → Ist“-Initialisierung und einem optionalen Vergleichsmodus. Das verhindert, dass die Timeline zur Sklavin der Hierarchie wird, und gibt ihr die Freiheit, die tatsächliche Produktionsrealität abzubilden.

7. Nächste Schritte (Empfohlene Tickets)

- T-Arch-1:** Datenmodell-Trennung — `StructureState` vs. `TimelineState` Typen definieren und Context auftrennen.
- T-Arch-2:** Transformations-Engine — Regeln für Plan→Ist-Initialisierung implementieren (`estimateDuration`, `initializeTimelineFromStructure`).
- T-Arch-3:** Cross-Scene Audio-Tracks — `plannedDuration` und `crossesIntoNextScenes` im Audio-Dropdown hinzufügen.

4. **T-Arch-4:** Compare/Sync-View Prototyp — Side-by-Side-Darstellung von Plan-Reihenfolge und Ist-Reihenfolge.

Erstellt aus der Pi-Analyse des Scriptony-Appwrite Repositories • Keine Code-Änderungen vorgenommen